

Министерство науки и высшего образования Российской Федерации
ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ

**«НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ
ИТМО»**

(Университет ИТМО)

Факультет **Прикладной информатики**

Образовательная программа **Мобильные и сетевые технологии**

Направление подготовки(специальность) **09.03.03 Прикладная информатика**

ЛАБОРАТОРНАЯ РАБОТА №6

По дисциплине «Проектирование и реализация баз данных»

Тема: Введение в СУБД MongoDB. Установка MongoDB. Начало работы с БД. Работа с БД в СУБД MongoDB.

Выполнил Остапишин Н.А. К3241.

Проверил Говорова М.М.

Санкт-Петербург 2026

1 Цель работы

- 1) Овладеть практическими навыками установки СУБД MongoDB.
- 2) Овладеть практическими навыками работы с CRUD-операциями, с вложенными объектами в коллекции базы данных MongoDB, агрегации и изменения данных, со ссылками и индексами в базе данных MongoDB.

2 Практические задания

Практическое задание для ЛР 6.1:

- Установите MongoDB для обоих типов систем (32/64 бита).
- Проверьте работоспособность системы запуском клиента mongo. -
Выполните методы:
 - a) `db.help()`
 - b) `db.help`
 - c) `db.stats()` - Создайте БД learn.
- Получите список доступных БД.
- Создайте коллекцию `unicorns`, вставив в нее документ `{name: 'Aurora', gender: 'f', weight: 450}`.
- Просмотрите список текущих коллекций.
- Переименуйте коллекцию `unicorns`.
- Просмотрите статистику коллекции.
- Удалите коллекцию.
- Удалите БД learn.

4 Выполнение лабораторной работы №6.1

4.2 Вызов базовых методов и создание БД

Для просмотра списка доступных методов объекта базы данных использовалась команда:

```
db.help()
```

Команда отображает основные функции и методы для работы с MongoDB.

```
learn> db.help()

Database Class:

getMongo           Returns the current database connection
getName           Returns the name of the DB
getCollectionNames Returns an array containing the names of all collections in the current d
atabase.
getCollectionInfos Returns an array of documents with collection information, i.e. collectio
n name and options, for the current database.
runCommand         Runs an arbitrary command on the database.
adminCommand       Runs an arbitrary command against the admin database.
aggregate          Runs a specified admin/diagnostic pipeline which does not require an unde
rlying collection.
getSiblingDB       Returns another database without modifying the db variable in the shell e
nvironment.
getCollection       Returns a collection or a view object that is functionally equivalent to
using the db.<collectionName>.
dropDatabase       Removes the current database, deleting the associated data files.
createUser         Creates a new user for the database on which the method is run. db.create
```

Рис. 1: Вызов метода db.help()

```
learn> db.stats()
{
  db: 'learn',
  collections: Long('0'),
  views: Long('0'),
  objects: Long('0'),
  avgObjSize: 0,
  dataSize: 0,
  storageSize: 0,
  indexes: Long('0'),
  indexSize: 0,
  totalSize: 0,
  scaleFactor: Long('1'),
  fsUsedSize: 0,
  fsTotalSize: 0,
  ok: 1
}
```

Рис. 2: Вызов метода db.stats()

Метод `stats()` используется для получения статистики о базе данных или коллекции.

Для создания базы данных использовалась команда:

```
use learn
```

Для просмотра списка всех существующих БД была использована команда:

```
show dbs
```

```
}  
learn> show dbs  
admin      40.00 KiB  
config     100.00 KiB  
local      40.00 KiB  
learn> |
```

Рис. 3: Переключение контекста на новую БД и выполнения команды show dbs

База данных learn ещё не отображалась в списке, поскольку MongoDB создает базы данных физически только после добавления первого документа или коллекции.

4.3 Работа с коллекциями и документами

Для создания коллекции unicorns была использована такая команда:

```
db.unicorns.insertOne({name: 'Aurora', gender: 'f', weight:  
450})
```

```
learn> db.unicorns.insertOne({name: 'Aurora', gender: 'f', weight: 450})  
{  
  acknowledged: true,  
  insertedId: ObjectId('6a188dda9488754d76abc114')  
}
```

Рис. 4: Создание коллекции unicorns

Для просмотра коллекций в БД была использована команда:

```
show collections
```

```
learn> show collections  
unicorns
```

Рис. 5: Получение списка коллекций

Для переименования коллекции была использована команда:

```
db.unicorns.renameCollection("horses")
```

```
Please enter a MongoDB connection string (default is mongodb://localhost:27020)  
learn> db.unicorns.renameCollection("horses")  
{ ok: 1 }  
learn> |
```

Рис. 6: Успешное переименование коллекции

Для получения статистики коллекции была использована команда:

```
db.horses.stats()
```

```
learn> db.horses.stats()
{
  ok: 1,
  capped: false,
  wiredTiger: {
    metadata: { formatVersion: 1 },
    creationString: 'access_pattern_hint=none,allocation_size=4KB,app_
metadata=(formatVersion=1),assert=(commit_timestamp=none,durable_time
stamp=none,read_timestamp=none,write_timestamp=off),block_allocation=be
st,block_compressor=snappy,block_manager=default,cache_resident=false,
checksum=on,colgroups=,collator=,columns=,dictionary=0,disaggregated=(
page_log=,storage_tier=none),encryption=(keyid=,name=),exclusive=false
,extractor=,format=btree,huffman_key=,huffman_value=,ignore_in_memory_
cache_size=false,immutable=false,import=(compare_timestamp=oldest_time
stamp,enabled=false,file_metadata=,metadata_file=,panic_corrupt=true,r
epair=false),in_memory=false,ingest=,internal_item_max=0,internal_key_
max=0,internal_key_truncate=true,internal_page_max=4KB,key_format=q,ke
y_gap=10,leaf_item_max=0,leaf_key_max=0,leaf_page_max=32KB,leaf_value_
max=64MB,log=(enabled=true),lsm=(auto_throttle=,bloom=,bloom_bit_count
=,bloom_config=,bloom_hash_count=,bloom_oldest=,chunk_count_limit=,chu
nk_max=,chunk_size=,merge_custom=(prefix=,start_generation=,suffix=),m
erge_max=,merge_min=),memory_page_image_max=0,memory_page_max=10M,os_c
```

Рис. 7: Успешное получение статистики коллекции

Для удаления коллекции была использована команда:

```
db.horses.drop()
```

```
learn> db.horses.drop()
true
```

Для удаления БД была использована команда:

```
db.dropDatabase()
```

```
learn> db.dropDatabase()
{ ok: 1, dropped: 'learn' }
learn>
```

Рис. 8: Успешное удаление коллекции и БД

5 Выполнение лабораторной работы №6.2

5.1 Вставка документов в коллекцию

Выполнение практического задания 2.1.1

С помощью команды use была создана БД learn. После этого, используя первый способ для добавления документов в коллекцию из описания ЛР 6.2, в коллекцию unicorns были добавлены документы:

```
db.unicorns.insertMany([
  {name: 'Horny', loves: ['carrot', 'papaya'], weight: 600, gender: 'm',
vampires: 63},
  {name: 'Aurora', loves: ['carrot', 'grape'], weight: 450, gender: 'f',
vampires: 43},
  {name: 'Unicrom', loves: ['energon', 'redbull'], weight: 984, gender:
'm', vampires: 182},
  {name: 'Roooooodles', loves: ['apple'], weight: 575, gender: 'm',
vampires: 99},
  {name: 'Solnara', loves: ['apple', 'carrot', 'chocolate'], weight:
550, gender: 'f', vampires: 80},
  {name: 'Ayna', loves: ['strawberry', 'lemon'], weight: 733, gender:
'f', vampires: 40},
  {name: 'Kenny', loves: ['grape', 'lemon'], weight: 690, gender: 'm',
vampires: 39},
  {name: 'Raleigh', loves: ['apple', 'sugar'], weight: 421, gender: 'm',
vampires: 2},
  {name: 'Leia', loves: ['apple', 'watermelon'], weight: 601, gender:
'f', vampires: 33},
  {name: 'Pilot', loves: ['apple', 'watermelon'], weight: 650, gender:
'm', vampires: 54},
  {name: 'Nimue', loves: ['grape', 'carrot'], weight: 540, gender: 'f'}
```

```
learn> db.unicorns.insertMany([
| {name: 'Horny', loves: ['carrot', 'papaya'], weight: 600,
| m', vampires: 63},
| {name: 'Aurora', loves: ['carrot', 'grape'], weight: 450,
| 'f', vampires: 43},
| {name: 'Unicrom', loves: ['energon', 'redbull'], weight:
gender: 'm', vampires: 182},
| {name: 'Roooooodles', loves: ['apple'], weight: 575, gend
vampires: 99},
| {name: 'Solnara', loves: ['apple', 'carrot', 'chocolate']
550, gender: 'f', vampires: 80},
| {name: 'Ayna', loves: ['strawberry', 'lemon'], weight: 73
'f', vampires: 40},
| {name: 'Kenny', loves: ['grape', 'lemon'], weight: 690, g
vampires: 39},
| {name: 'Raleigh', loves: ['apple', 'sugar'], weight: 421,
'm', vampires: 2},
| {name: 'Leia', loves: ['apple', 'watermelon'], weight: 60
'f', vampires: 33},
| {name: 'Pilot', loves: ['apple', 'watermelon'], weight: 6
gender: 'm', vampires: 54},
| {name: 'Nimue', loves: ['grape', 'carrot'], weight: 540,
'f'}
| ])
{
  acknowledged: true,
  insertedIds: {
    '0': ObjectId('6a188f1a9488754d76abc120'),
  }
}
```

Рис. 9: Добавление документа в коллекцию первым способом
И также используя второй способ из описания, был добавлен еще один документ:

5.2 Выборка данных из БД

В ходе выполнения задания были сформированы запросы выборки документов из коллекции `unicorns` с использованием метода `find()`.

1) Получение списка самцов:

```
learn> db.unicorns.find({gender: 'm'}).sort({name: 1})
[
  {
    _id: ObjectId('6a188eee9488754d76abc115'),
    name: 'Horny',
    loves: [ 'carrot', 'papaya' ],
    weight: 600,
    gender: 'm',
    vampires: 63
  },
  {
    _id: ObjectId('6a188f1a9488754d76abc120'),
    name: 'Horny',
    loves: [ 'carrot', 'papaya' ],
    weight: 600,
    gender: 'm',
    vampires: 63
  },
  {
    _id: ObjectId('6a188eee9488754d76abc11b'),
```

Рис. 12: Список всех самцов с сортировкой по имени

Запрос выполняет выборку всех самцов единорогов и сортирует результат по имени в алфавитном порядке.

Выполнение практического задания 2.2.2

Модифицируйте запрос для вывода списков самцов единорогов, исключив из результата информацию о предпочтениях.

```
learn> db.unicorns.find({gender: 'm'}, {loves: 0, gender: 0})
[
  {
    _id: ObjectId('6a188eee9488754d76abc115'),
    name: 'Horny',
    weight: 600,
    vampires: 63
  },
  {
    _id: ObjectId('6a188eee9488754d76abc117'),
    name: 'Unicrom',
    weight: 984,
    vampires: 182
  },
  {
    _id: ObjectId('6a188eee9488754d76abc118'),
```

Рис. 17: Список самцов, исключая информацию о предпочтениях

Выполнение практического задания 2.2.3

Вывести список единорогов в обратном порядке добавления.

```
learn> db.unicorns.find().sort({$natural:-1})
[
  {
    _id: ObjectId('6a188f1a9488754d76abc12a'),
    name: 'Nimue',
    loves: [ 'grape', 'carrot' ],
    weight: 540,
    gender: 'f'
  },
  {
    _id: ObjectId('6a188f1a9488754d76abc129'),
    name: 'Pilot',
    loves: [ 'apple', 'watermelon' ],
    weight: 650,
    gender: 'm',
    vampires: 54
  },
  {
    _id: ObjectId('6a188f1a9488754d76abc128'),
    name: 'Leia',
    loves: [ 'apple', 'watermelon' ],
    weight: 601,
    gender: 'f',
    vampires: 33
  },
  {
    _id: ObjectId('6a188f1a9488754d76abc127'),
    name: 'Raleigh',

```

Рис. 18: Список единорогов в обратном порядке добавления

Выполнение практического задания 2.2.4

Вывести список единорогов с названием первого любимого предпочтения, исключив идентификатор.

```
learn> db.unicorns.find({}, {loves: {$slice: 1}, _id: 0})
[
  {
    name: 'Horny',
    loves: [ 'carrot' ],
    weight: 600,
    gender: 'm',
    vampires: 63
  },
  {
    name: 'Aurora',
    loves: [ 'carrot' ],
    weight: 450,
    gender: 'f',
    vampires: 43
  },
]
```

Рис. 19: Список единорогов с названием первого предпочтения, исключая поле `_id`

В ходе выполнения задания была выполнена выборка документов из коллекции `unicorns` с использованием проекции полей.

Для массива `loves` был применен оператор `$slice`, позволяющий вывести только первый элемент массива предпочтений. Также из результата был исключен идентификатор `_id`.

5.3 Логические операторы

Выполнение практического задания 2.3.1

Вывести список самок единорогов весом от полутонны до 700 кг, исключив вывод идентификатора.

```
learn> db.unicorns.find({gender: 'f', weight: {$gte: 500, $lte: 700}},
{_id: 0})
[
  {
    name: 'Solnara',
    loves: [ 'apple', 'carrot', 'chocolate' ],
    weight: 550,
    gender: 'f',
    vampires: 80
  },
  {
    name: 'Leia',
    loves: [ 'apple', 'watermelon' ],
    weight: 601,
    gender: 'f',
    vampires: 33
  }
]
```

Рис. 20: Список единорогов весом от 500 кг до 700 кг, исключая поле `_id`

В ходе выполнения задания были использованы логические операторы сравнения `$gte` и `$lte`.

Выполнение практического задания 2.3.2

Вывести список самцов единорогов весом от полутонны и предпочитающих grape и lemon, исключив вывод идентификатора.

```
learn> db.unicorns.find({gender: 'm', weight: {$gte: 500}, loves: {$all: [ 'grape', 'lemon' ]}}, {_id: 0})
[
  {
    name: 'Kenny',
    loves: [ 'grape', 'lemon' ],
    weight: 690,
    gender: 'm',
    vampires: 39
  },
  {
    name: 'Kenny',
    loves: [ 'grape', 'lemon' ],
    weight: 690,
    gender: 'm',
    vampires: 39
  }
]
learn> |
```

Рис. 21: Список самцов весом от 500 кг и предпочитающих grape и lemon, исключая поле `_id`

В ходе выполнения задания был использован оператор `$all`, предназначенный для поиска документов, массив которых содержит все указанные элементы.

Выполнение практического задания 2.3.3

Найти всех единорогов, не имеющих ключ vampires.

```
learn> db.unicorns.find({vampires: {$exists: false}})
[
  {
    _id: ObjectId('6a188eee9488754d76abc11f'),
    name: 'Nimue',
    loves: [ 'grape', 'carrot' ],
    weight: 540,
    gender: 'f'
  },
  {
    _id: ObjectId('6a188f1a9488754d76abc12a'),
    name: 'Nimue',
    loves: [ 'grape', 'carrot' ],
    weight: 540,
    gender: 'f'
  }
]
```

Рис. 22: Список единорогов, не имеющих поле vampires

В ходе выполнения задания был использован оператор `$exists`, позволяющий проверить наличие или отсутствие поля в документе.

Выполнение практического задания 2.3.4

Вывести список упорядоченный список имен самцов единорогов с информацией об их первом предпочтении.

```
learn> db.unicorns.find({gender: 'm'}, {_id: 0, name: 1, loves: {$slice: 1}}).sort({name: 1})
[
  { name: 'Horny', loves: [ 'carrot' ] },
  { name: 'Horny', loves: [ 'carrot' ] },
  { name: 'Kenny', loves: [ 'grape' ] },
  { name: 'Kenny', loves: [ 'grape' ] },
  { name: 'Pilot', loves: [ 'apple' ] },
  { name: 'Pilot', loves: [ 'apple' ] },
  { name: 'Raleigh', loves: [ 'apple' ] },
  { name: 'Raleigh', loves: [ 'apple' ] },
  { name: 'Roooooodles', loves: [ 'apple' ] },
  { name: 'Roooooodles', loves: [ 'apple' ] },
  { name: 'Unicrom', loves: [ 'energon' ] },
  { name: 'Unicrom', loves: [ 'energon' ] }
]
learn> |
```

Рис. 23: Список самцов с информацией об их первом предпочтении, сортировка по имени

5.4 Запросы к вложенным объектам

Выполнение практического задания 3.1.1

- 1) В ходе выполнения задания была создана коллекция `towns`, содержащая вложенные документы `mayor`. Были добавлены сведения о городах, населении, известных особенностях и мэрах.

```
learn> db.towns.insertOne({name: "Punxsutawney", population: 6200, last_s  
ensus: ISODate("2008-01-31"), famous_for: [""], mayor: {name: "Jim Wehrle"}}  
)  
{  
  acknowledged: true,  
  insertedId: ObjectId('6a18a0929488754d76abc12b')  
}  
learn> db.towns.insertOne({name: "New York", population: 22200000, last_s  
ensus: ISODate("2009-07-31"), famous_for: ["statue of liberty", "food"], may  
or: {name: "Michael Bloomberg", party: "I"}})  
{  
  acknowledged: true,  
  insertedId: ObjectId('6a18a0bb9488754d76abc12c')  
}  
learn> db.towns.insertOne({name: "Portland", population: 528000, last_s  
ensus: ISODate("2009-07-20"), famous_for: ["beer", "food"], mayor: {name: "S  
am Adams", party: "D"}})  
{  
  acknowledged: true,  
  insertedId: ObjectId('6a18a0c09488754d76abc12d')  
}  
learn> |
```

Рис. 24: Создание коллекции towns и добавление документов

- 2) *Сформировать запрос, который возвращает список городов с независимыми мэрами (party="I"). Вывести только название города и информацию о мэре.*

```
learn> db.towns.find({"mayor.party": "I"}, {_id: 0, name: 1, mayor: 1})  
[  
  { name: 'New York', mayor: { name: 'Michael Bloomberg', party: 'I' } }  
]  
learn> |
```

Рис. 25: Список городов с независимыми мэрами

В ходе выполнения задания был сформирован запрос к вложенному объекту с использованием оператора точки. Были выбраны города, мэры которых являются независимыми (party = "I").

3) Сформировать запрос, который возвращает список беспартийных мэров (`party` отсутствует) . Вывести только название города и информацию о мэре.

```
learn> db.towns.find({"mayor.party": {$exists: false}}, {_id: 0, name: 1, ma
mayor: 1})
[ { name: 'Punxsutawney', mayor: { name: 'Jim Wehrle' } } ]
```

Рис. 26: Список городов с беспартийными мэрами

В ходе выполнения задания был использован оператор `$exists` для поиска документов, в которых отсутствует поле `party` у вложенного объекта `mayor`.

Выполнение практического задания 3.1.2

- 1) Сформировать функцию для вывода списка самцов единорогов.
- 2) Создать курсор для этого списка из первых двух особей с сортировкой в лексикографическом порядке.
- 3) Вывести результат, используя `forEach`

```
learn> var cursor = db.unicorns.find({gender: "m"}).sort({name: 1}).limit(2);

learn> cursor.forEach(function(obj) { print(obj.name); })
Horny
Horny
```

5.5 Агрегированные запросы

Выполнение практического задания 3.2.1

Вывести количество самок единорогов весом от полутонны до 600 кг.

```
learn> db.unicorns.countDocuments({gender: "f", weight: {$gte: 500, $lte: 600}})
4
```

Рис. 30: Вывод количества самок единорогов весом от 500 кг и до 600 кг

Выполнение практического задания 3.2.2

Вывести список уникальных предпочтений единорогов.

```
learn> db.unicorns.distinct("loves")
[
  'apple',      'carrot',
  'chocolate', 'energon',
  'grape',      'lemon',
  'papaya',     'redbull',
  'strawberry', 'sugar',
  'watermelon'
]
```

Рис. 31: Вывод списка уникальных предпочтений единорогов

В ходе выполнения задания был использован метод `distinct()` для получения уникальных значений предпочтений единорогов.

Выполнение практического задания 3.2.3

Посчитать количество особей единорогов обоих полов.


```
learn> db.unicorns.aggregate([{$group: {_id: "$gender", count: {$sum: 1}}}]
[ { _id: 'm', count: 12 }, { _id: 'f', count: 10 } ]
```

Рис. 32: Вывод количества самцов и самок

В ходе выполнения задания был использован агрегатный запрос `aggregate()` с оператором `$group` для подсчета количества документов по полю `gender`.

5.6 Редактирование данных

Выполнение практического задания 3.3.2

1. Для самки единорога *Ayna* внести изменения в БД: теперь ее вес 800, она убила 51 вампира.
2. Проверить содержимое коллекции `unicorns`.

```
learn> db.unicorns.updateOne({name: 'Ayna'}, {$set: {weight: 800, vampires:
51}})
{
  acknowledged: true,
  insertedId: null,
  matchedCount: 1,
  modifiedCount: 1,
  upsertedCount: 0
}
```

Рис. 33: Изменение документа и проверка содержимого

В ходе выполнения задания был использован оператор `$set` для изменения отдельных полей документа.

Выполнение практического задания 3.3.3

1. Для самца единорога *Raleigh* внести изменения в БД: теперь он любит рэдбул.
2. Проверить содержимое коллекции `unicorns`.

```
learn> db.unicorns.updateOne({name: 'Raleigh'}, {$push: {loves: 'redbull'}})
{
  acknowledged: true,
  insertedId: null,
  matchedCount: 1,
  modifiedCount: 1,
  upsertedCount: 0
}
```

Рис. 34: Изменение документа и проверка содержимого

В ходе выполнения задания был использован оператор `$push` для добавления нового элемента в массив `loves`.

Выполнение практического задания 3.3.4

1. *Всем самцам единорогов увеличить количество убитых вампиров на 5.*
2. *Проверить содержимое коллекции unicorns.*

```
learn> db.unicorns.updateMany({gender: 'm'}, {$inc: {vampires: 5}})
{
  acknowledged: true,
  insertedId: null,
  matchedCount: 12,
  modifiedCount: 12,
  upsertedCount: 0
}
```

Рис. 35: Изменение документа и проверка содержимого

В ходе выполнения задания был использован оператор `$inc` для увеличения числового значения поля `vampires`.

Выполнение практического задания 3.3.5

1. *Изменить информацию о городе Портланд: мэр этого города теперь беспартийный.*
2. *Проверить содержимое коллекции towns.*

```
learn> db.towns.updateOne({name: 'Portland'}, {$unset: {"mayor.party": 1}})
{
  acknowledged: true,
  insertedId: null,
  matchedCount: 1,
  modifiedCount: 1,
  upsertedCount: 0
}
```

Рис. 36: Изменение документа и проверка содержимого

В ходе выполнения задания был использован оператор `$unset` для удаления поля `party` из вложенного объекта `mayor`.

Выполнение практического задания 3.3.6

1. *Изменить информацию о самце единорога Pilot: теперь он любит и шоколад.*
2. *Проверить содержимое коллекции unicorns.*

```

learn> db.unicorns.updateOne({name: 'Pilot'}, {$push: {loves: 'chocolate'}})
{
  acknowledged: true,
  insertedId: null,
  matchedCount: 1,
  modifiedCount: 1,
  upsertedCount: 0
}
learn> db.unicorns.find({name: 'Pilot'})
[
  {
    _id: ObjectId('6a188eee9488754d76abc11e'),
    name: 'Pilot',
    loves: [ 'apple', 'watermelon', 'chocolate' ],
    weight: 650,
    gender: 'm',
    vampires: 59
  },
  {
    _id: ObjectId('6a188f1a9488754d76abc129'),
    name: 'Pilot',
    loves: [ 'apple', 'watermelon' ],
    weight: 650,
    gender: 'm',
    vampires: 59
  }
]

```

Рис. 37: Изменение документа и проверка содержимого

В ходе выполнения задания был использован оператор `$push` для добавления нового элемента в массив предпочтений.

Выполнение практического задания 3.3.7

1. Изменить информацию о самке единорога Aurora: теперь она любит еще и сахар, и лимоны.
2. Проверить содержимое коллекции `unicorns`.

```
learn> db.unicorns.updateOne({name: 'Aurora'}, {$addToSet: {loves: {$each: [
'sugar', 'lemon']}}})
{
  acknowledged: true,
  insertedId: null,
  matchedCount: 1,
  modifiedCount: 1,
  upsertedCount: 0
}
learn> db.unicorns.find({name: 'Aurora'})
[
  {
    _id: ObjectId('6a188eee9488754d76abc116'),
    name: 'Aurora',
    loves: [ 'carrot', 'grape', 'sugar', 'lemon' ],
    weight: 450,
    gender: 'f',
    vampires: 43
  }
]
```

Рис. 38: Изменение документа и проверка содержимого

В ходе выполнения задания был использован оператор `$addToSet` совместно с `$each` для добавления нескольких уникальных элементов в массив.

5.7 Удаление данных

Выполнение практического задания 3.4.1

- 1) Удаление городов с беспартийными мэрами

```
learn> db.towns.deleteMany({"mayor.party": "I"})
{ acknowledged: true, deletedCount: 1 }
learn> |
```

Рис. 39: Удаление данных и проверка содержимого

- 2) Очистка коллекции towns

Рис. 40: Очистка коллекции и проверка содержимого

- 3) Просмотр коллекций

```
learn> db.towns.find()
[
  {
    _id: ObjectId('6a18a0929488754d76abc12b'),
    name: 'Punxsutawney',
    population: 6200,
    last_sensus: ISODate('2008-01-31T00:00:00.000Z'),
    famous_for: [ '' ],
    mayor: { name: 'Jim Wehrle' }
  },
  {
    _id: ObjectId('6a18a0c09488754d76abc12d'),
    name: 'Portland',
    population: 528000,
    last_sensus: ISODate('2009-07-20T00:00:00.000Z'),
    famous_for: [ 'beer', 'food' ],
    mayor: { name: 'Sam Adams' }
  }
]
```

Рис. 41: Получение списка коллекций

В ходе выполнения задания была выполнена полная очистка коллекции towns и просмотр списка существующих коллекций базы данных.

5.8 Ссылки в MongoDB

Выполнение практического задания 4.1.1

- 1) *Создайте коллекцию зон обитания единорогов, указав в качестве идентификатора кратко название зоны, далее включив полное название и описание.*
- 2) *Включите для нескольких единорогов в документы ссылку на зону обитания, используя второй способ автоматического связывания.*
- 3) *Проверьте содержание коллекции единорогов.*

1) Создание коллекции habitats

```
learn> db.habitats.insertMany([
|   { _id: "forest", full_name: "Magic Forest", description: "Forest habitat
|   for unicorns" },
|   { _id: "mountains", full_name: "Crystal Mountains", description: "Mountai
|   n habitat for unicorns" },
|   { _id: "lake", full_name: "Silver Lake", description: "Lake habitat for u
|   nicorns" }
| ])
{
  acknowledged: true,
  insertedIds: { '0': 'forest', '1': 'mountains', '2': 'lake' }
}
```

Рис. 42: Создание коллекции habitats

В ходе выполнения задания была создана коллекция `habitats`, содержащая информацию о зонах обитания единорогов.

2) Добавление ссылок

```
learn> db.unicorns.updateOne({name: "Horny"}, {$set: {habitat: {$ref: "habitats", $id: "forest"}}})
{
  acknowledged: true,
  insertedId: null,
  matchedCount: 1,
  modifiedCount: 1,
  upsertedCount: 0
}
learn> db.unicorns.updateOne({name: "Aurora"}, {$set: {habitat: {$ref: "habitats", $id: "mountains"}}})
{
  acknowledged: true,
  insertedId: null,
  matchedCount: 1,
  modifiedCount: 1,
  upsertedCount: 0
}
```

Рис. 43: Добавление ссылок для двух единорогов

3) Проверка содержимого коллекции

```
learn> db.unicorns.find({}, {name: 1, habitat: 1})
[
  {
    _id: ObjectId('6a188eee9488754d76abc115'),
    name: 'Horny',
    habitat: DBRef('habitats', 'forest')
  },
  {
    _id: ObjectId('6a188eee9488754d76abc116'),
    name: 'Aurora',
    habitat: DBRef('habitats', 'mountains')
  },
  {
    _id: ObjectId('6a188eee9488754d76abc117'), name: 'Unicrom' },
  {
    _id: ObjectId('6a188eee9488754d76abc118'), name: 'Roooooodles' }
]
```

Рис. 44: Проверка содержимого коллекции unicorns В ходе выполнения задания были созданы ссылки между коллекциями с использованием механизма DBRef.

5.9 Настройка индексов

Выполнение практического задания 4.2.1

- 1) Проверьте, можно ли задать для коллекции `unicorns` индекс для ключа `name` с флагом `unique`.

```
learn> db.unicorns.createIndex({name: 1}, {unique: true})
name_1
```

Рис. 45: Успешное создание индекса

5.9 Управление индексами

Выполнение практического задания 4.3.1

- 1) Получите информацию о всех индексах коллекции `unicorns`.
 - 2) Удалите все индексы, кроме индекса для идентификатора.
 - 3) Попробуйте удалить индекс для идентификатора.
- 1) Получение информации обо всех индексах

```
learn> db.unicorns.getIndexes()
[
  { v: 2, key: { _id: 1 }, name: '_id_' },
  { v: 2, key: { name: 1 }, name: 'name_1', unique: true }
]
```

Рис. 46: Получение списка индексов

2) Удаление всех индексов, кроме `_id`

Рис. 47: Удаление индексов и получение списка индексов

В ходе выполнения задания были удалены все пользовательские индексы коллекции `unicorns`. После удаления в коллекции остался только системный индекс `_id`

```
learn> db.unicorns.dropIndex("name_1")
{ nIndexesWas: 2, ok: 1 }
```

5.9 План запроса

Выполнение практического задания 4.4.1

- 1) *Создайте объемную коллекцию `numbers`, задействовав курсор:*

```
for(i = 0; i < 100000; i++){db.numbers.insert({value: i})}
```
- 2) *Выберите последних четыре документа.*
- 3) *Проанализируйте план выполнения запроса 2. Сколько потребовалось времени на выполнение запроса? (по значению параметра `executionTimeMillis`)*
- 4) *Создайте индекс для ключа `value`.*
- 5) *Получите информацию о всех индексах коллекции `numbers`.*
- 6) *Выполните запрос 2.*
- 7) *Проанализируйте план выполнения запроса с установленным индексом. Сколько потребовалось времени на выполнение запроса?*
- 8) *Сравните время выполнения запросов с индексом и без. Дайте ответ на вопрос: какой запрос более эффективен?*

1) Создание объемной коллекции `numbers`

```
test> use learn
switched to db learn
learn> for(i = 0; i < 100000; i++){db.numbers.insertOne({value: i})}
{
  acknowledged: true,
  insertedId: ObjectId('6a18b284ba426489b8ad47b3')
}
learn> |
```

Рис. 49: Создание коллекции `numbers`

2) Выборка последних четырех документов


```
learn> db.numbers.find().sort({value: -1}).limit(4)
[
  { _id: ObjectId('6a18b284ba426489b8ad47b3'), value: 99999 },
  { _id: ObjectId('6a18b284ba426489b8ad47b2'), value: 99998 },
  { _id: ObjectId('6a18b284ba426489b8ad47b1'), value: 99997 },
  { _id: ObjectId('6a18b284ba426489b8ad47b0'), value: 99996 }
]
```

Рис. 50: Получение последних 4 документов

В ходе выполнения задания был выполнен запрос для получения последних четырех документов коллекции numbers.

3) Анализ плана выполнения запроса без индекса

```
learn> db.numbers.find().sort({value: -1}).limit(4).explain("executionStats")
{
  explainVersion: '1',
  queryPlanner: {
    namespace: 'learn.numbers',
    parsedQuery: {},
    indexFilterSet: false,
    queryHash: 'AE5753F2',
    planCacheShapeHash: 'AE5753F2',
    planCacheKey: 'AE69D0D4',
    optimizationTimeMillis: 0,
    maxIndexedOrSolutionsReached: false,
    maxIndexedAndSolutionsReached: false,
    maxScansToExplodeReached: false,
    prunedSimilarIndexes: false,
    winningPlan: {
      isCached: false,
      stage: 'SORT',
      sortPattern: { value: -1 }
    }
  }
}
```

Рис. 51: Получение анализа запроса без индекса

В поле executionTimeMillis значение 128, значит время выполнения запроса составило: 128ms.

4) Создание индекса по полю value

```
learn> db.numbers.createIndex({value: 1})
value_1
learn> |
```

Рис. 52: Создание индекса по полю value

5) Получение информации об индексах

```
value_1
learn> db.numbers.getIndexes()
[
  { v: 2, key: { _id: 1 }, name: '_id_' },
  { v: 2, key: { value: 1 }, name: 'value_1' }
]
learn> |
```

Рис. 53: Получение списка индексов

6) Повторное выполнение запроса

```
learn> db.numbers.find().sort({value: -1}).limit(4)
[
  { _id: ObjectId('6a18b284ba426489b8ad47b3'), value: 99999 },
  { _id: ObjectId('6a18b284ba426489b8ad47b2'), value: 99998 },
  { _id: ObjectId('6a18b284ba426489b8ad47b1'), value: 99997 },
  { _id: ObjectId('6a18b284ba426489b8ad47b0'), value: 99996 }
]
```

Рис. 54: Повторное выполнение запроса

7) Анализ плана запроса с индексом

```
learn> db.numbers.find().sort({value: -1}).limit(4).explain("executionStats")
{
  explainVersion: '1',
  queryPlanner: {
    namespace: 'learn.numbers',
    parsedQuery: {},
    indexFilterSet: false,
    queryHash: 'AE5753F2',
    planCacheShapeHash: 'AE5753F2',
    planCacheKey: 'AE69D0D4',
    optimizationTimeMillis: 0,
    maxIndexedOrSolutionsReached: false,
    maxIndexedAndSolutionsReached: false,
    maxScansToExplodeReached: false,
    prunedSimilarIndexes: false,
    winningPlan: {
      isCached: false,
      stage: 'LIMIT',
      limitAmount: 4,
      inputStage: {
        stage: 'FETCH',
        nss: 'learn.numbers',
        inputStage: {
          stage: 'IXSCAN',
          nss: 'learn.numbers',
          keyPattern: { value: 1 },
          indexName: 'value_1',
          isMultiKey: false,
          multiKeyPaths: { value: [] },
          isUnique: false,
          isSparse: false,
          isPartial: false
        }
      }
    }
  }
}
```

Рис. 55: Получение анализа запроса с индексом

Теперь в поле `executionTimeMillis` значение 0, значит время выполнения запроса составило: 0ms за счет работы индекса.

Более эффективным является запрос с использованием индекса, так как MongoDB выполняет поиск по индексированной структуре данных без полного просмотра всей коллекции.

Вывод

В результате выполнения лабораторной работы освоены основные операции MongoDB: создание БД и коллекций, вставка и выборка документов с фильтрацией, сортировкой и проекцией, работа с массивами и вложенными объектами. Применены логические операторы, агрегация, обновление и удаление данных. Реализованы ссылки между коллекциями, индексы и анализ планов запросов. В связи с обновлением версии mongosh использованы современные методы: insertOne, updateOne, deleteMany, createIndex.